



Bharatiya Vidya Bhavan
Sardar Patel Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai)

Munshi Nagar, Andheri (West), Mumbai – 400 058.

Department of Computer Engineering

Academic Year: 2025-26

Aim: Integration of R and Python in Power BI

Objective

- Understand integration of R and Python in Power BI
 - Perform data import, visualization, and analytics
 - Apply basic machine learning models
-

Requirements

- Power BI Desktop
 - R (installed)
 - Python (Anaconda recommended)
-

Task 1: Configuration

Steps:

1. Open Power BI Desktop
2. Go to File → Options → Options
3. Select R scripting → Set R path
4. Select Python scripting → Set Python path
5. Click OK

Output:

R and Python configured successfully

Task 2: Data Import using Python

Steps:

1. Click Get Data → Python Script
2. Enter:

```
import pandas as pd
```

```
data = pd.DataFrame({  
    'Hours_Studied':[1,2,3,4,5],  
    'Marks':[35,50,65,70,90]  
})
```

```
data
```

3. Click OK → Load

Task:

Display dataset

Task 3: Data Import using R**Steps:**

1. Click Get Data → R Script
2. Enter:

```
data <- data.frame(  
  Hours_Studied=c(1,2,3,4,5),  
  Marks=c(35,50,65,70,90)  
)  
data
```

3. Load data

Task:

Verify dataset

Task 4: Python Visual**Steps:**

1. Select Python Visual
2. Drag fields
3. Enter:

```
import matplotlib.pyplot as plt  
plt.scatter(dataset['Hours_Studied'], dataset['Marks'])  
plt.show()
```

Task:

Interpret graph

Task 5: R Visual**Steps:**

1. Select R Visual
2. Drag fields
3. Enter:

```
library(ggplot2)  
ggplot(dataset, aes(x=Hours_Studied, y=Marks)) + geom_point()
```

Task:

Compare with Python output

Task 6: Machine Learning using Python**Steps:**

1. Go to Transform Data → Run Python Script
2. Enter:

```
from sklearn.linear_model import LinearRegression  
  
df = dataset
```

```
X = df[['Hours_Studied']]
y = df['Marks']

model = LinearRegression()
model.fit(X, y)

df['Predicted'] = model.predict(X)

df
```

Task:

Plot Actual vs Predicted

Task 7: Clustering using Python**Steps:**

```
from sklearn.cluster import KMeans

df = dataset
kmeans = KMeans(n_clusters=2)
df['Cluster'] = kmeans.fit_predict(df[['Hours_Studied', 'Marks']])

df
```

Task:

Visualize clusters

Task 8: Time Series using R**Steps:**

```
library(forecast)

ts_data <- ts(dataset$Marks)
model <- auto.arima(ts_data)
forecast(model, h=3)
```

Task:

Plot forecast

Task 9: Data Cleaning (Python)**Steps:**

```
df = dataset

df = df.dropna()
df['Performance'] = df['Marks'] * 1.1

df
```

Task:

Analyze cleaned data

Task 10: 3D Plot using Python in Power BI

Steps:

1. Click on **Python Visual** in Power BI
2. Drag fields: Hours_Studied, Marks, Performance (or any 3 numeric fields)
3. Enter the following code:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(dataset['Hours_Studied'], dataset['Marks'], dataset['Performance'])

ax.set_xlabel('Hours Studied')
ax.set_ylabel('Marks')
ax.set_zlabel('Performance')

plt.show()
```

Task:

- Observe 3D relationship between variables
 - Rotate the visual and interpret patterns
-

Task 11: 3D Plot using R in Power BI

Steps:

1. Click on **R Visual**
2. Drag 3 numeric fields
3. Enter the following code:

```
library(scatterplot3d)

scatterplot3d(dataset$Hours_Studied,
dataset$Marks,
dataset$Performance,
color="blue",
pch=19)
```

Task:

- Compare with Python 3D plot
-

Task 12: 3D Interactive Plot using Python (Plotly)

Steps:

1. Click on Python Visual
2. Drag fields: Hours_Studied, Marks, Performance
3. Enter the following code:

```
import plotly.express as px
```

```
fig = px.scatter_3d(dataset,  
x='Hours_Studied',  
y='Marks',  
z='Performance',  
color='Marks')
```

```
fig.show()
```

Task:

- Rotate and zoom the 3D plot
 - Observe color variation based on Marks
 - Compare with static matplotlib 3D plot
-

Task 13: 3D Interactive Plot using R (Plotly)

Steps:

1. Click on R Visual
2. Drag fields: Hours_Studied, Marks, Performance
3. Enter the following code:

```
library(plotly)
```

```
plot_ly(data = dataset,  
x = ~Hours_Studied,  
y = ~Marks,  
z = ~Performance,  
type = "scatter3d",  
mode = "markers",  
color = ~Marks)
```

Task:

- Interact with the 3D plot (rotate, zoom)
 - Compare with Python Plotly output
-

Conclusion

Students learned how to integrate R and Python with Power BI for:

- Data import
- Visualization (2D & 3D)
- Machine Learning
- Advanced analytics